Importance of Tuning Hyperparameters of Machine Learning Algorithms

Hilde J.P. Weerts*
Eindhoven University of Technology
The Netherlands

Andreas C. Müller
Columbia University
New York, U.S.A.

Joaquin Vanschoren
Eindhoven University of Technology
The Netherlands

arXiv:2007.07588v1 [cs.LG] 15 Jul 2020

ABSTRACT

The performance of many machine learning algorithms depends on their hyperparameter settings. The goal of this study is to determine whether it is important to tune a hyperparameter or whether it can be safely set to a default value. We present a methodology to determine the importance of tuning a hyperparameter based on a non-inferiority test and *tuning risk*: the performance loss that is incurred when a hyperparameter is not tuned, but set to a default value. Because our methods require the notion of a default parameter, we present a simple procedure that can be used to determine reasonable default parameters. We apply our methods in a benchmark study using 59 datasets from OpenML. Our results show that leaving particular hyperparameters at their default value is non-inferior to tuning these hyperparameters. In some cases, leaving the hyperparameter at its default value even outperforms tuning it using a search procedure with a limited number of iterations.

KEYWORDS

machine learning, hyperparameter tuning, meta learning

1 INTRODUCTION

Many parameters of machine learning algorithms are derived through training. Additionally, most modern machine learning algorithms have parameters that need to be fixed before running them. Such parameters are referred to as *hyperparameters* [7, 8, 12, 15]. In many cases, the performance of an algorithm on a given learning task depends on its hyperparameter settings. In order to obtain the best performance, machine learning practitioners can *tune* the hyperparameters. Methods for tuning hyperparameters for the problem at hand require the definition of a search space: the set of hyperparameters and ranges that need to be considered. Additionally, one needs to define the search heuristic that is used to search for good hyperparameters within the search space. Tuning is generally a computationally expensive procedure that becomes more expensive as the search space increases. Currently, there does not exist empirical evidence on which hyperparameters are important to tune and which hyperparameters result in similar performance when set to a reasonable default value. Hyperparameters that fall into the latter category might be eliminated from the search space entirely when computational resources are limited. In this work we study the importance of tuning hyperparameters and aim to provide empirical evidence of hyperparameters that might be eliminated from the search space.

We present a methodology to determine which hyperparameters are important to tune, based on empirical performance data derived from experiments across multiple datasets. Because our methodology requires the notion of a default parameter setting, we introduce a simple procedure to deduce reasonable default parameters from performance data. We apply our approach in a benchmark study of two popular classification algorithms: random forests and support vector machines. In particular, we analyze the importance of hyperparameter tuning based on the performance of these algorithms across 59 datasets that were taken from the OpenML-CC18 benchmark suite.

Our results suggest that leaving particular hyperparameters at our computed default value leads to non-inferior performance. Moreover, given a random search approach with a limited number of iterations, fixing the hyperparameter to our default value sometimes outperforms tuning all hyperparameters at once. Machine learning practitioners might want to leave these hyperparameters at their default value, rather than invoking an expensive random search procedure. For other hyperparameters, we observed a difference between the fixed and non-fixed conditions. This indicates that these hyperparameters should not be left at our computed default value.

The remainder of this work is structured as follows. In Section 2, we provide a short overview of related work. In Section 3, we present our default parameter determination procedure as well as a methodology for determining the importance of tuning. Section 4 covers details of our benchmark experiments, including the used datasets, algorithms, hyperparameters, performance measures, and search strategy. In Section 5, the results of the experiments are presented. Finally, the conclusions and limitations are discussed in Section 6.

2 RELATED WORK

2.1 Hyperparameter Default Values

In the work of Probst et al. [12], default values are determined using surrogate models as follows. First, for each of the 38 binary classification tasks in the study, a surrogate model that predicts the performance of the algorithm based on hyperparameter settings is learned from empirical performance data. Second, a large number of hyperparameter configurations are sampled and their performance is estimated using the surrogate models. Finally, the default parameters are determined by minimizing the average risk. In contrast to this method, we will use a simple, intuitive heuristic to determine default parameters directly from the performance data.

2.2 Hyperparameter Importance

The importance of hyperparameter tuning has been studied in the work of Mantovani et al. [8]. The authors use properties of the

*Work done while intern at Columbia University in 2018.

learning task, known as meta-features, to predict whether hyperparameter tuning will lead to better results than models obtained using default parameter settings. Additionally, Lavesson and Davidsson [7] show that tuning hyperparameters is often more important than the choice of the machine learning algorithm. In contrast to these works, we study the importance of tuning particular hyperparameters, rather than the importance of tuning at all.

Van Rijn and Hutter [15] study the importance of particular hyperparameters as well as interactions between hyperparameters. The authors define important hyperparameters as parameters that explain most variance in performance across multiple datasets. The explained variance is determined through a functional ANOVA framework. A limitation of this approach is that the results do not directly translate into guidelines on which particular hyperparameters are important to tune. For example, a hyperparameter for which one specific setting always performs well is considered important, as it explains variance in performance. However, such a parameter does not require extensive tuning. Instead, it can simply be set to the setting that always results in good performance.

This issue is alleviated by Probst et al. [12], who introduce the *tunability* of a hyperparameter: the performance gain that can be achieved by tuning the hyperparameter. The authors compare the performance of leaving all hyperparameters at their default values to tuning the hyperparameter of interest while leaving all other hyperparameters at their defaults. Their approach can be used directly to determine the importance of tuning a hyperparameter.

The approach taken in this work is similar to the work of Probst et al. [12]. As opposed to the aforementioned work, we are interested in the *tuning risk* of a hyperparameter: the performance loss that is incurred when a hyperparameter is set to a default value instead of being tuned, while all other hyperparameters are tuned. Probst et al. [12] describe the method used in this work and mention that "the two alternatives can be seen as similar to forward and backward selection of variables in stepwise regression" (p. 7). The difference between the two approaches is reflected in the way default parameters are chosen. The default parameters determined by Probst et al. [12] are chosen in a joint fashion, while the default parameters used in this work are determined univariately. That is, Probst et al. [12] determine a specific set of default hyperparameter values at the same time, whereas we determine the default setting of a hyperparameter irrespective of the values of other hyperparameters. Consequently, the approach of Probst et al. [12] takes into account possible interaction effects between hyperparameters. This is beneficial in their forward selection approach, as most of the hyperparameters are left at their default value. In our backward selection approach, however, potential interaction effects are optimized by tuning the interacting hyperparameter, allowing us to choose the default parameters using a univariate approach. In future research, these interaction effects can be investigated further.

3 METHODS

3.1 Notation

Let A be an algorithm with H hyperparameters with domains $\Theta_1, ..., \Theta_h$ and configuration space $\boldsymbol{\Theta} = \Theta_1 \times ... \times \Theta_h$. Let $\boldsymbol{\theta} = \langle \theta_1, ..., \theta_h \rangle$ with $\theta_i \in \Theta_i$ denote a setting of all hyperparameters of A.

The performance of an algorithm with hyperparameter settings $\boldsymbol{\theta}$ on a specific learning task can be expressed as a loss function $L(Y, \hat{f}(X, \boldsymbol{\theta}))$. Here, Y is the target variable, X a vector of features and $f(., \boldsymbol{\theta})$ the predictive model resulting from training algorithm A with hyperparameter settings $\boldsymbol{\theta}$. For example, classification accuracy can be expressed as the mean of a 0/1 loss function defined as follows:

$$L(y, \hat{f}(x, \boldsymbol{\theta})) = \begin{cases} 0, & \text{if } y = \hat{f}(x, \boldsymbol{\theta}) \\ 1, & \text{otherwise} \end{cases} \tag{1}$$

Usually, one is interested in estimating the expected loss [12]. Let Let $R_j(\boldsymbol{\theta})$ denote the expected loss (risk) of a model trained with hyperparameter settings $\boldsymbol{\theta}$ on dataset j. Let θ_i^j denote the default setting of hyperparameter i for dataset j. Moreover, let $R_j(\theta_i^j)$ denote the risk for dataset j of fixing hyperparameter i to θ_i^j while tuning all other $H - 1$ hyperparameters. Finally, let θ_i^{*j} denote the best setting for parameter i for dataset j, i.e.:

$$\theta_i^{*j} = \arg\min_{\theta_i \in \Theta_i} R_j(\theta_i) \tag{2}$$

Given an algorithm A and hyperparameter i, we aim to determine 1) a reasonable default setting θ_i^j, for $i = 1, ..., h$ and $j = 1, ..., m$, and 2) whether $R_j(\theta_i^j)$ is non-inferior to $R_j(\theta_i^{*j})$ across datasets.

3.2 Hyperparameter Default Values

The default setting problem can be formalized as follows. Given

- an algorithm A with configuration space $\boldsymbol{\Theta}$
- a hyperparameter i with domain Θ_i
- M datasets $\mathcal{D}^{(1)}, ..., \mathcal{D}^{(m)}$
- for each dataset j, a set of K empirical risk measurements with different hyperparameter settings $\langle R_j(\boldsymbol{\theta}_k) \rangle_{k=1}^K$, where $\boldsymbol{\theta}_k \in \boldsymbol{\Theta}$

we aim to determine a default setting θ_i^j for hyperparameter i, for dataset j.

3.2.1 Approach. We use a simple heuristic procedure to determine default values. The intuition behind our method is to find a setting that, across many datasets, most often results in good performance.

It is important to note that if we were to use a single default value in the hyperparameter importance experiment, we leak information on the best hyperparameter value of a specific dataset. Therefore, we determine a default setting using a leave-one-out approach. That is, we determine θ_i^j using only performance data related to the other $M - 1$ datasets.

First, we take a subset of all empirical performance measurements that represent good performance. That is, for each dataset, we pick the n hyperparameter settings that resulted in the best performance, i.e. lowest risk. Note that the top n performance data consists of $M \cdot n$ empirical performance measurements. This is similar to the first step of the prior distribution estimation method proposed by Van Rijn and Hutter [15]. The choice of n is potentially important. On the one hand, picking n too small might give a large weight to outliers. On the other hand, picking n too large could result in the inclusion of bad performance measurements in the subset.

Second, we determine which parameter setting occurs most frequently within this subset. The intuition is that a setting that worked well on most datasets is a good candidate for a default value. Moreover, if there exists a single setting that works best for most datasets, our approach is certain to find it. Note that this would not be the case when using, for example, the median. For hyperparameters with a nominal or boolean domain, the setting that worked well most often is simply the mode. Since hyperparameter settings are picked randomly across the domain, parameters with a continuous domain or large integer domain, e.g. more than 50 possible values, must first be discretized. In this work, we will determine the bin size b using the Freedman-Diaconis rule [3], presented in Equation 3.

$$b = 2\frac{IQR}{\sqrt[3]{N}} \tag{3}$$

where IQR is the interquartile range of the data and N is the number of observations. The Freedman-Diaconis rule is known to be resilient to outliers. An alternative of this rule is Sturges formula [14]. The automatic histogram binning, as implemented in the python library `numpy` [10], uses the maximum of the Freedman-Diaconis rule and Sturges formula. However, Sturges formula assumes that the data follows a Gaussian distribution. Since we have no reason to assume that this holds for our data, we only use the Freedman-Diaconis rule.

The advantage of our method is that it is very simple and intuitive. In future work, it would be interesting to compare our simple, univariate heuristic with more sophisticated methods, such as default parameter estimation through surrogate models [12].

3.2.2 Meta-feature dependent default values. In the open source machine learning library scikit-learn [11], the default values of some hyperparameters depend on properties (or meta-features) of the dataset. In particular, `max_features` of the random forest and `gamma` of the support vector machine depend on the number of features in the dataset. The default values of all other hyperparameters in scikit-learn that are involved in this study are not meta-feature dependent. As there is no indication of what functions might be reasonable for these hyperparameters, we will only apply a meta-feature dependent approach for `max_features` and `gamma`. Let p denote the number of features of a dataset. Then, the default parameters are $\sqrt{p}$ and $1/p$ for `max_features` and `gamma` respectively. For these hyperparameters, we investigate an alternative default parameter estimation approach. Rather than choosing the value that occurs most frequently, we use non-linear least squares to fit several functions to the top 10 performance data, as implemented in the `curve_fit` function of the python library *scipy* [5].

3.3 Tuning Risk of Hyperparameters

In this section, we explain the experiment design and methods that are used to determine the importance of tuning a hyperparameter.

3.3.1 Experiment Design. We adhere to a repeated measures experiment design, where the unit of analysis is a dataset. The dependent variable in our study is the cross validated *performance* of an instantiation of an algorithm. The two independent variables in our study are *condition* and *random search seed*. The *condition* variable indicates whether a hyperparameter is *fixed* or *non-fixed*. That is, for some algorithm A, hyperparameter i, and dataset j, we have:

- *Fixed:* fix hyperparameter i to default setting θ_i^j and tune all other $H-1$ hyperparameters;
- *Non-fixed:* tune hyperparameter i as well as all other $H-1$ hyperparameters.

The hyperparameters will be tuned using a random search strategy [1] within a nested cross validation procedure. In the outer cross validation loop, we split the data into training and test sets. In the inner cross validation loop, we split each training set further into training and validation sets. In each iteration of the random search, the average validation set performance is determined to assess the performance of a hyperparameter setting. The best hyperparameter setting is used to determine the test set performance. The final performance of the algorithm is the average test set performance over all outer folds.

Although the random seeds are the same for fixed and non-fixed conditions, they are not shared among datasets. For example, the first random seed of task 1 is not necessarily identical to the first random seed of task 2.

3.3.2 Approach. We evaluate the results of the experiment through both an absolute measure (tuning risk) and a statistical measure (non-inferiority).

Tuning Risk - To understand to what extent the *condition* affects the performance of the algorithm, we compute the difference in risk between the fixed and non-fixed conditions. Analogous to the *tunability* measure introduced by Probst et al. [12], we define *tuning risk* of hyperparameter i for dataset j and seed s as follows:

$$d_{i,j,s} = R(\theta_i^j) - R(\theta_i^{*j,s}) \tag{4}$$

The tuning risk quantifies the risk of not tuning a particular hyperparameter. Larger values of $d_{i,j,s}$ indicate that the risk of the fixed condition is higher than the risk of the non-fixed condition. To summarize the differences over all M datasets and seeds, we need an aggregating function. An obvious function is to compute the simple average. Other examples would be to use the median or some weighting scheme, e.g. based on properties of the dataset. Note that using the median would give more weight to intermediate risk values. Because we do not have evidence for the justification of a particular weighting scheme, we compute the simple average tuning risk of hyperparameter i:

$$d_i = \frac{1}{S \cdot M} \sum_{s=1}^{S} \sum_{j=1}^{M} d_{i,j,s} \tag{5}$$

We also investigate the standard deviation of $d_{i,j,s}$, which is denoted by s_i.

An assumption of this definition of tuning risk is that every unit of risk is equally relevant. In practice, however, this is often not the case. For example, when considering the missclassification rate, an increase from 0.01 to 0.02, might be considered worse than an increase from 0.49 to 0.50, as in the former case the number of misclassified instances is doubled. Hence, it depends on the learning task whether the absolute tuning risk is a relevant improvement or not. To alleviate this issue, we define the *relative tuning risk*

as the relative difference in risk between the fixed and non-fixed conditions. The measure is formalized as follows:

$$d^R_{i,j,s} = \frac{R(\theta^j_i) - R(\theta^{*j,s}_i)}{R(\theta^{*j,s}_i)} \tag{6}$$

Again, we aggregate the relative tuning risk by computing the average:

$$d^R_i = \frac{1}{S \cdot M} \sum_{s=1}^{S} \sum_{j=1}^{M} d^R_{i,j,s} \tag{7}$$

Additionally, we compute the standard deviation of $d^R_{i,j,s}$, which we will denote by s^R_i.

Non-inferiority - We use the statistical measure of non-inferiority to determine whether the performance of the fixed condition is comparable to the performance of the non-fixed condition. This measure differs from traditional hypothesis testing, where the goal is to show that one group is different from another group with respect to some variable. Instead, we wish to determine whether an effect observed in one group is non-inferior to the effect in another group. Two one-sided non-inferiority tests can be combined into a method referred to as equivalence testing. This method is often applied in clinical and psychological research [6, 17]. For example, one might want to determine whether some new treatment has similar benefits compared to an existing treatment.

An important consideration in non-inferiority studies is the non-inferiority margin: the maximum difference in effect size that is considered to be irrelevant in practice. The non-inferiority margin is also known as smallest effect size of interest. It is more meaningful to define relevant improvements in performance on a relative scale than on an absolute scale. Therefore, we will only investigate the relative tuning risk in a non-inferiority test. We are mostly interested in the case where the observed risk in the fixed condition is higher than the risk observed in the non-fixed condition. We formulate our research question as follows:

> *Is the observed risk in the fixed condition non-inferior to the risk observed in the non-fixed condition?*

To answer our research question, we perform a non-parametric one sided test of non-inferiority for paired samples. We use a one-sided version of the Two One-Sided Tests (TOST) procedure described in the work of Mara and Cribbie [9]. The authors show that the non-parametric TOST procedure is more powerful for non-normal distributions than the TOST procedure that uses a paired t-test, because the former is less sensitive to outliers. Since we have no reason to assume that our data follows a normal distribution, we use the non-parametric alternative. The procedure is very similar to the Wilcoxon signed ranks tests, the biggest difference being the inversion of the null and alternative hypotheses. In the remainder of this section, we describe the procedure in more detail.

We define the relative risk of observations as $\frac{x_f - x_{nf}}{x_{nf}}$, where x_f denotes an observation of the fixed condition and x_{nf} an observation of the non-fixed condition. Let δ be the non-inferiority margin for the relative risk. Additionally, let M_{rr} be the population median of the relative risk. Then the null and alternative hypotheses are as follows:

$$H_0 : M_{rr} \geq \delta \tag{8}$$

$$H_1 : M_{rr} < \delta \tag{9}$$

We compute signed ranks for observations $\frac{x_f - x_{nf}}{x_{nf}} - \delta$. Let s_{nr} denote the absolute value of the sum of the negative ranks and N the number of observations. Then the test statistic z is defined as follows:

$$z = \frac{s_{nr} - \left(\frac{N(N+1)}{4}\right)}{\sqrt{\frac{N(N+1)(2N+1)}{24}}} \tag{10}$$

For a single test, H_0 is rejected if $z \geq z_{1-\alpha}$, where $z_{1-\alpha}$ is the value of the standard normal distribution at significance level α. The experiment will be performed multiple times in this study, each time for a different hyperparameter. We use the Holm-Bonferroni method [4] to control the family-wise error.

4 EXPERIMENT DETAILS

In this section we give an overview of the datasets, algorithms, performance measures, and experiment setups. All experiments are executed using dockerized Python applications[1] on the Azure Cloud Computing platform.

4.1 Datasets

We use data from the open machine learning environment OpenML [16]. In particular, we use datasets from the OpenML-CC18, a curated machine learning benchmark suite of 73 classification datasets. A description of these datasets can be found in Appendix A.1. In this study, we used 59 of these 73 datasets. One dataset was not included in this study because of technical issues arising from the large number of nominal features in this dataset. Additionally, due to time constraints, we limited the training time of a single algorithm run on a dataset to 3 hours. Datasets for which more than 10% of the default value experiment runs lasted more than 3 hours are left out of the analysis. The 14 datasets that are excluded from the study are listed in Appendix A.2. It is important to note that all excluded datasets had a relatively high number of features, instances, or both. This might bias our results towards smaller datasets when considering meta-feature dependent default values.

4.2 Algorithms and Hyperparameters

In this work we consider two algorithms as implemented in scikit-learn. We consider a Support Vector Machine (SVM) with a Radial Basis Function (RBF) kernel and the Random Forest. The parameter ranges that are considered are taken from the automatic machine learning package auto-sklearn [2]. The ranges for SVM and random forest can be found in Table 1 and Table 2 respectively.

The complete pipeline consists of simple pre-processing steps and the algorithm. The pre-processing pipeline includes missing value imputation by the mean (continuous features) or mode (categorical features), one hot encoding for categorical features, and removal of features with zero variance. Because SVM's are known

[1] See https://github.com/hildeweerts/hyperimp.

Table 1: Hyperparameter ranges for `SVC`.

Hyperparameter	Type	Range
gamma	continuous	$[2^{-15}, 2^{3}]$ (log-scale)
C	continuous	$[2^{-}5, 2^{1}5]$ (log-scale)
tol	continuous	$[10^{-5}, 10^{-1}]$ (log-scale)
shrinking	boolean	{True, False}
kernel	discrete	rbf

Table 2: Hyperparameter ranges for `RandomForestClassifier`. Parameters annotated with * are fixed to a value different than the default of scikit-learn.

Hyperparameter	Type	Range
bootstrap	boolean	{True, False}
criterion	nominal	{gini, entropy}
max_features	continuous	[0,1]
min_samples_leaf	integer	[1, 20]
min_samples_split	integer	[2, 20]
n_estimators*	integer	500

to be sensitive to different feature scales, a scaling step is added to the SVM pre-processing pipeline.

4.3 Performance Measures

We compare two performance measures, as implemented in OpenML, at the default parameter determination stage: accuracy and macro-averaged Area under the ROC Curve (AUC). In macro-averaging, the measure is computed locally over each category first, then the average over all categories is taken, weighted by the number of instances of that class. The macro-averaged AUC is computed using the approach of Provost and Domingos [13]. In this approach, the ROC curve of each class is constructed by comparing the class of interest to the union of all other classes. Note that these curves can be sensitive to changes in prevalence of the classes within the union. At this point, multi-class AUC is not yet implemented in scikit-learn. Therefore, we will use accuracy as a performance measure in the random search procedure of the hyperparameter importance experiment.

4.4 Experiment 1: Default Values

In order to determine good default values, we need empirical risk measurements $\langle R(\boldsymbol{\theta}_k)\rangle_{k=1}^{K}$ for each of the M datasets. In this experiment $M = 59$ and we choose $K = 1000$. In other words, for each dataset, we evaluate 1,000 random configurations $\boldsymbol{\theta}_i$ of each algorithm, i.e. 59,000 evaluations per algorithm. Each hyperparameter will be picked uniformly at random from the corresponding hyperparameter range and scale. The performance of a configuration is calculated using 10-fold cross validation. Recall that we use the best n settings for each task to determine the default parameters. In accordance to the work of Van Rijn and Hutter [15], we pick $n = 10$.

4.5 Experiment 2: Importance of Tuning

We use nested cross validation to determine the performance of both the fixed and non-fixed conditions. In the outer loop, we use 10-fold cross validation to split the data in training and test sets. In the inner loop, we use 5-fold cross validation to split the training data further into training and validation sets. For each fold in the inner loop, a random search strategy with 100 iterations is applied to tune hyperparameters. Note that the hyperparameter settings that are used to determine test set performance can be different for each of the 10 folds in the outer cross validation. We repeat the experiment $S = 10$ times, using a different seed for the random search each time.

5 RESULTS

In this section we discuss the results of our experiments. First, we discuss the results of the default value experiment. Second, we discuss the results from the hyperparameter tuning importance experiments. All performance data that was used in this study is publicly available on OpenML[2].

5.1 Experiment 1: Default Values

In this section, we first discuss the collected performance data. Subsequently, we discuss the computed default values. In particular, we compare our computations with the default values computed by Probst et al. [12] and the default values used in scikit-learn. Finally, we discuss the meta-feature dependent default parameters.

5.1.1 Performance data. After selecting the top 10 performance data, we visualized the distributions of the hyperparameter settings in a histogram, as illustrated in Figure 1. From the distribution depicted in Figure 1a, we can conclude that fixing `min_samples_leaf` to 1 resulted in good performance most often. For most hyperparameters, the distribution of accuracy- and AUC-based data is very similar. For others, one may arrive at a different default parameter depending on the performance measure that was used. For instance, consider the distribution of `C` shown in Figure 1b. An accuracy-based default value for `C` would be in the order of magnitude 1, whereas an AUC-based default value would be in the order of magnitude 4. The histograms of all other hyperparameters can be found in Appendix B.

5.1.2 Default values. Recall that the default values are computed for each task separately, using a leave-one-out approach. The average and standard deviation of the computed default values across all datasets are displayed in Table 3.

For most hyperparameters, the standard deviation of the computed default values across datasets is zero or relatively small, considering the scaling. For example, the default values of `C` using an accuracy-based approach range between 3 and 11, with a standard deviation of 3.224. This is negligible on a logarithmic scale. The distribution of default values with non-zero variance are displayed in Appendix C.

The differences between defaults resulting from the accuracy and AUC approach are in line with the histograms shown in the previous section. For `bootstrap`, `criterion`, `min_samples_split`, `C`, `tol` we find different default parameters than the ones currently used in

[2]See https://www.openml.org/s/98/.

Table 3: Average/Mode (numerical/categorical) and standard deviation of our calculated defaults over all datasets, defaults of scikit-learn, and defaults as calculated by Probst et al. [12]. Hyperparameters that are not computed in the work of Probst et al. [12] (e.g. because they do not exist in the R implementation) are indicated with N/A. The standard deviation that is shown for `max_features`, annotated with *, is the standard deviation of the exponent. The standard deviation that is shown for boolean variables, annotated with **, is the standard deviation when considering `True` as 1 and `False` as 0. The standard deviation shown for `criterion`, annotated with *, is the standard deviation when considerint `entropy` as 1 and `gini` as 0.**

Hyperparameter	Average/Mode (accuracy)	Standard deviation (accuracy)	Average/Mode (AUC)	Standard deviation (AUC)	scikit-learn	Probst et al. [12]
Random Forest						
bootstrap	False	**0	False	**0	True	False
criterion	entropy	***0	entropy	***0	gini	N/A
max_features	n^0.737	*0.009	n^0.714	*0.006	sqrt(n)	0.284
min_samples_leaf	1	0	1	0	1	0
min_samples_split	5	0	2.3	0.8	2	N/A
SVM						
C	8.888	3.224	16646	63	1.000	475.504
gamma	1.2 e-02	1.5 e-04	8.1 e-03	1.5 e-03	1/n	5 e-03
shrinking	True	**0	True	**0.36	True	N/A
tol	4.6 e-05	8.8 e-08	5.4 e-02	1.3 e-02	1.0 e-03	N/A

scikit-learn, although it should be noted that the default parameter of `min_samples_split` in scikit-learn was returned often by the AUC-based approach. For `bootstrap` and `gamma`, we find similar values as Probst et al. [12]. On the other hand, the default values for `C` differ a lot between the accuracy-based defaults, AUC-based defaults, the default used in scikit-learn, and the defaults computed by Probst et al. [12].

5.1.3 Meta-feature dependent hyperparameters. As stated in Section 3, we will determine the hyperparameters for `max_features` and `gamma` using an approach that considers the number of features of a dataset. Figure 2 shows a scatter plot of the hyperparameter value versus the number of features, including the functions that that were fitted. The Root Mean Squared Error (RMSE), R^2, Root Mean Squared Logarithmic Error (RMSLE), and logarithmic R^2 (LR^2) for each of the functions can be found in TableZ4.

From Figure 2a, it becomes clear that for our datasets, the default value in scikit-learn, $\sqrt{p}$, seems to underestimate `max_features`. In fact, on a linear scale, it performs worse than all other functions that we considered, whereas $m = 0.16^p$ performs best. However, RMSE and R^2 are slighty biased, because there are relatively few large datasets. When we consider the RSMLE and LR^2 instead, $m = p^{0.74}$ performs best, followed by $m = \sqrt{p}$. In Figure 2b, we do not observe a clear pattern for `gamma`. Additionally, we observe negative values for both R^2 and LR^2 for all functions in Table 4. The default value in scikit-learn, $1/p$, performs slightly better than the other two functions.

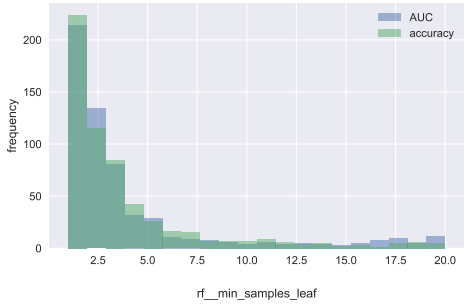
In experiment 2, where we investigate the importance of tuning, we will consider $\sqrt{p}$ and $p^{0.74}$ for `max_features`, and $1/p$ and 0.006 for `gamma`.

Table 4: RMSE, R^2, RMSLE, and LR^2 of several functions that depend on the number of features of the dataset, fitted on the accuracy-based top 10 performance data for `max_features` and `gamma`.

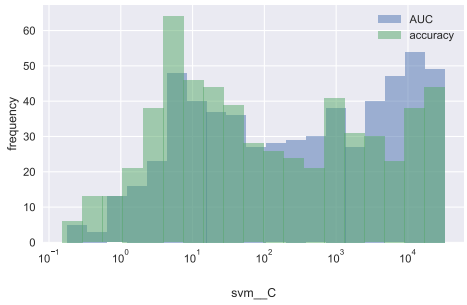
function	RMSE	R^2	RSMLE	LR^2
max_features				
m = 0.16*p	21.64	0.77	0.90	0.13
m = p ^0.74	23.13	0.73	0.73	0.42
m = 1.15^sqrt(p)	24.33	0.70	1.24	-0.66
m = sqrt(p)	41.38	0.14	0.81	0.28
gamma				
m = 0.00574*p	1.25	-0.07	0.46	-0.12
m = 1/p	1.24	-0.05	0.44	-0.04
m = 0.006	1.27	-0.09	0.46	-0.14

5.2 Experiment 2: Importance of Tuning

In order to investigate the tuning process, we visualize the average accuracy and rank of the fixed and non-fixed conditions at each iteration. As an example, Figure 3 depicts the maximum average validation set accuracy observed up until a certain iteration of the random search for `gamma`, using our computed default value. Figure 4 depicts the average rank over the number of iterations for `max_features`, using our computed default value. The ranks are computed as follows. For each measurement, for each iteration, the condition with the highest validation accuracy receives rank 1 and the other rank 2. The visualizations of the other hyperparameter experiments can be found in Appendix D and Appendix E.

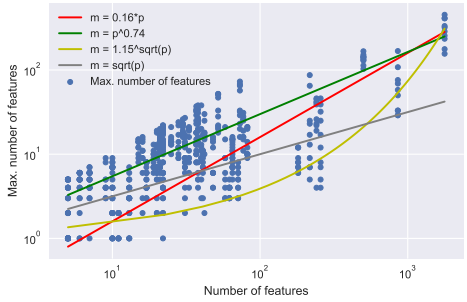


(a) Random Forest - `min_samples_leaf`

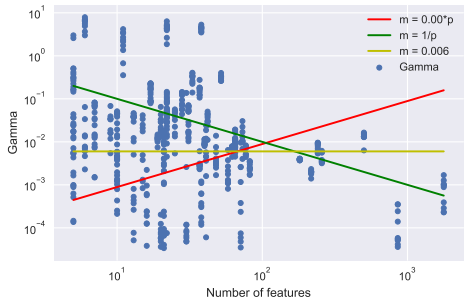


(b) SVM - `C`

Figure 1: The distribution of `min_samples_leaf` and `C` within the top 10 highest performing hyperparameter settings per dataset, derived from either accuracy or AUC-based performance data.



(a) Random Forest - `max_features`



(b) SVM - `gamma`

Figure 2: Meta-feature dependent parameter values of the top 10 performance data (accuracy-based only) against the number of features (p) of the corresponding dataset.

For all hyperparameters except SVM's `shrinking`, the average rank of the fixed condition is lower than the average rank of the non-fixed condition in the first iteration. For many hyperparameters, the fixed condition, on average, outperforms the non-fixed condition for all 100 iterations. This indicates that 100 iterations was not enough to find the best possible hyperparameter settings of the algorithm. For SVM's `shrinking` and `tol`, we observe that the average rank is consistently close to 1.5, which indicates that neither the fixed nor the non-fixed condition was superior. For random forest's `max_features` (using the computed default value), SVM's `C`, SVM's `gamma` (both computed default value and scikit-learn's default value), the non-fixed condition, on average, outperforms the fixed condition after 15 to 30 iterations.

5.2.1 Tuning Risk. The average and standard deviation of both tuning risk and relative tuning risk are shown in Table 5. Additionally, Figure 5 shows the distribution of tuning risk for each of the hyperparameters. There exist three datasets for which the non-fixed SVM almost always resulted in perfect accuracy: OpenML task 11, 49, and 10093. Because relative tuning risk is undefined when the non-fixed risk is equal to 0, these tasks are left out of the tuning risk analysis. Note that the tuning risk is often negative. A negative tuning risk indicates that the hyperparameters in the non-fixed condition have not been tuned to their optimal value. If they were, the performance of the non-fixed condition should always be at least

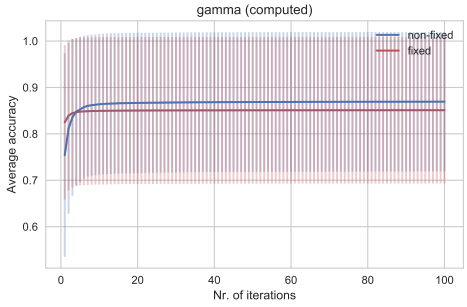


Figure 3: Average accuracy (+/- standard deviation) of the the fixed and non-fixed conditions over number of iterations for hyperparameter `gamma` (computed default setting).

as good as the performance of the fixed condition, as an exhaustive search would include the default value of the fixed condition. This is related to the length of our random search procedure, which we limited to 100 iterations. In future research, this issue could be alleviated by increasing the number of iterations or using a smarter search procedure such as the prior based approach suggested by [15].

From Table 5, we observe that for `bootstrap`, `criterion`, `min_samples_leaf`, `min_samples_split`, `shrinking`, and `tol`, both

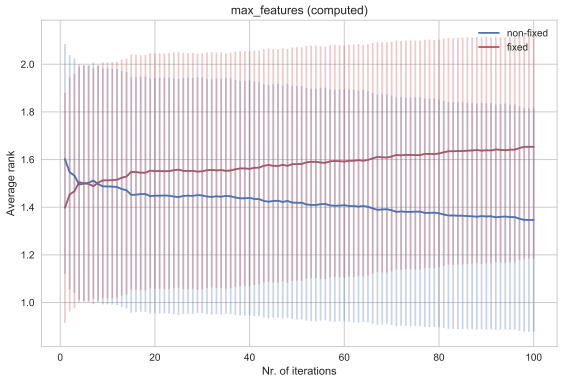


Figure 4: Average rank (+/- standard deviation) of the the fixed and non-fixed conditions over number of iterations for hyperparameter `max_features` (computed default setting).

Table 5: Average and standard deviation of tuning risk (d_i, s_i) and relative tuning risk (d_i^R, s_i^R).

Hyperparameter	d_i	s_i	d_i^R	s_i^R
Random Forest				
bootstrap	-0.0002	0.0053	-0.0053	0.0820
criterion	-0.0002	0.0041	0.0035	0.0793
max_features	0.0011	0.0068	0.0742	0.2822
max_features (scikit-learn)	0.0010	0.0134	0.0309	0.2037
min_samples_leaf	-0.0030	0.0077	-0.0505	0.1280
min_samples_split	-0.0009	0.0048	-0.0110	0.0829
SVM				
C	0.0008	0.0062	0.0252	0.2415
gamma	0.0180	0.0599	0.4595	1.4687
gamma (scikit-learn)	0.0039	0.0115	0.0503	0.2498
shrinking	0.0000	0.0005	-0.0003	0.0057
tol	-0.0001	0.0016	0.0006	0.0179

the tuning risk and relative tuning risk are close to or lower than zero. This is in line with the average ranks shown in the previous section.

In Figure 5a, we observe a few high tuning risk scores for `gamma` (computed default) and `max_features` (scikit-learn default). This indicates that there exist a few datasets for which the fixed condition performed much worse than the non-fixed condition. This effect is even larger for `gamma` (computed default) when considering relative tuning risk. The tuning risk of `gamma` when using scikit-learn's default parameter is much better than the tuning risk of `gamma` when using our non meta-feature dependent default setting. This indicates that it is important to take into account the number of features when optimizing `gamma`.

For `max_features`, our computed default value resulted in slightly better tuning risk but worse relative tuning risk compared to using scikit-learn's default value. In particular, there are several outliers with a relative tuning risk higher than 2. This is not as expected, given the analysis on meta-feature dependent parameter values in Section 5.1.3. It indicates that the top 10 performance data gives a decent representation of absolute performance, but not necessarily of relative performance. This might be caused by the fact that all data points are weighted equally when determining the default value, disregarding the fact that some learning tasks are much easier than others.

5.2.2 Non-inferiority test. The test statistics and p-values associated with non-inferiority tests with a non-inferiority margin of $\delta = 0.01$ are shown in Table 6. To determine which null hypotheses are rejected, we used the Holm-Bonferroni method with a significance level of $\alpha = 0.05$.

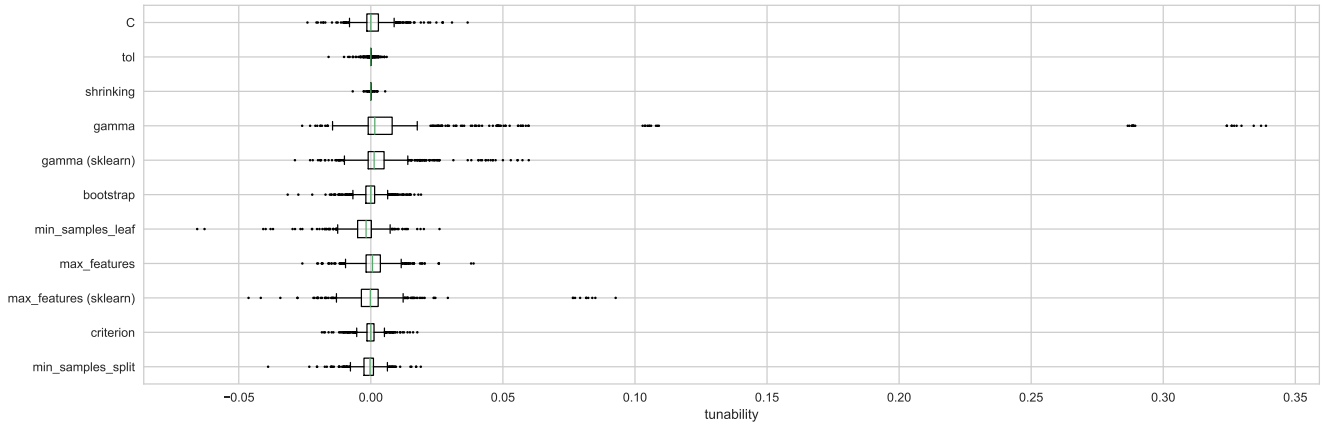
Table 6: Test statistics, p-values, and conclusions of non-inferiority tests. Conclusions are obtained through the Holm-Bonferroni method [4].

Hyperparameter	z	p	H_0
Random Forest			
bootstrap	7.93	1.11 e-15	rejected
criterion	8.16	1.11 e-16	rejected
max_features	-2.02	9.78 e-01	not rejected
max_features (scikit-learn)	3.59	1.63 e-04	rejected
min_samples_leaf	15.04	0.00 e+00	rejected
min_samples_split	10.06	0.00 e+00	rejected
SVM			
C	1.83	3.40 e-02	not rejected
gamma	-8.24	1.00 e+00	not rejected
gamma (scikit-learn)	-4.61	1.00 e+00	not rejected
shrinking	20.49	0.00 e+00	rejected
tol	16.32	0.00 e+00	rejected

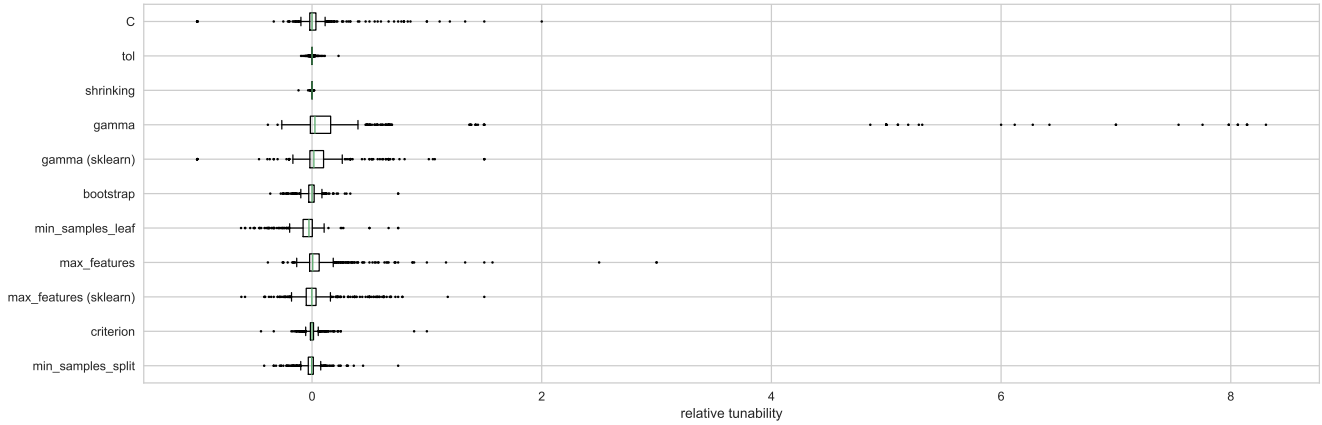
The results show that for all hyperparameters, except for `max_features` (computed default), `C`, and `gamma` (both computed and scikit-learn default) the relative risk is not more than 1% higher in the fixed condition compared to the non-fixed condition. This indicates that for these hyperparameters, setting the hyperparameter to the default value is non-inferior to tuning the parameter through a random search using 100 iterations. For `C`, `gamma`, and `max_features` (computed default) this is not the case.

6 CONCLUSIONS

In this study, we have first presented a simple heuristic to find default hyperparameters. In our experiments, we touched upon determining meta-feature dependent default parameters. Additionally, we presented a methodology for determining the importance of tuning a hyperparameter empirically. In contrast to previous work, our approach can be used to determine the loss incurred when one of the hyperparameters is not tuned but set to a default value. This is different from the study of Probst et al. [12], who investigate the performance gain of leaving all hyperparameters to a default value and tuning one hyperparameter. We have applied



(a) Tuning risk



(b) Relative tuning risk

Figure 5: The distribution of tuning risk and relative tuning risk measurements ($d_{i,j,r}$) per hyperparameter.

our methodology in a benchmark study using 59 different datasets. In this way, we provide empirical evidence that can be consulted by machine learning practitioners before they start a computationally and time intensive tuning process.

Our results show that using our computed default value often results in non-inferior performance compared to tuning the hyperparameter. It should be noted that the number of iterations turned out to be too low to tune several configurations of the algorithms to the best possible performance in the non-fixed condition. Although this indicates that our default parameters were reasonable, the number of iterations could be increased in future work. For other hyperparameters, such as random forest's `max_features` and SVM's `gamma` and `C`, we observed a high tuning risk and relative tuning risk, which suggests that it is important to tune these hyperparameters.

A limitation of our work is that the default parameters are only determined once and are not validated separately. As a result, it is unclear how our simple default parameter estimation method affects the results of the second experiment. In particular, it is unclear whether our choice of n results in a good representation of hyperparameters with good performance. This could be resolved in future work by comparing different methods for finding default parameters. Moreover, it would be interesting to compare the effect of particular performance measures, e.g. accuracy and AUC, on computed default values.

Another limitation is that we excluded several datasets from the OpenML-CC18 that turned out to have a high number of instances and/or features. It is unclear how this affects our conclusions, in particular for the determination of meta-feature dependent default values. A related issue is that it is unclear whether the OpenML-CC18 is a good representation of datasets that machine learning practitioners encounter in real life. Our conclusions might not hold for datasets that are very different from the ones analyzed in this work.

Finally, it is important to note that we have only considered leaving a single hyperparameter at a default value. In future work, interactions between hyperparameters could be further investigated.

ACKNOWLEDGMENTS

We thank Microsoft Azure for providing the computational resources for this study.

REFERENCES

[1] James Bergstra and Yoshua Bengio. 2012. Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research* 13 (2012), 281–305. http://dl.acm.org/citation.cfm?id=2188395

[2] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and Robust Automated Machine Learning. In *Advances in Neural Information Processing Systems 28*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett (Eds.). Curran Associates, Inc., 2962–2970. http://papers.nips.cc/paper/5872-efficient-and-robust-automated-machine-learning.pdf

[3] David Freedman and Persi Diaconis. 1981. On the histogram as a density estimator:L2 theory. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete* 57, 4 (01 Dec 1981), 453–476. https://doi.org/10.1007/BF01025868

[4] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. http://www.jstor.org/stable/4615733

[5] Eric Jones, Travis Oliphant, Pearu Peterson, et al. 2001. SciPy: Open source scientific tools for Python. (2001). http://www.scipy.org/ [Online; accessed June 25, 2018].

[6] DaniÃńl Lakens. 2017. Equivalence Tests. *Social Psychological and Personality Science* 8, 4 (may 2017), 355–362. https://doi.org/10.1177/1948550617697177

[7] Niklas Lavesson and Paul Davidsson. 2006. Quantifying the Impact of Learning Algorithm Parameter Tuning. In *Proceedings of the 21st National Conference on Artificial Intelligence - Volume 1 (AAAI'06)*. AAAI Press, 395–400. http://dl.acm.org/citation.cfm?id=1597538.1597602

[8] Rafael G. Mantovani, Andre L. D. Rossi, Joaquin Vanschoren, Bernd Bischl, and Andre C. P. L. F. Carvalho. 2015. To tune or not to tune: Recommending when to adjust SVM hyper-parameters via meta-learning. In *2015 International Joint Conference on Neural Networks (IJCNN)*. IEEE. https://doi.org/10.1109/ijcnn.2015.7280644

[9] Constance A. Mara and Robert A. Cribbie. 2012. Paired-Samples Tests of Equivalence. *Communications in Statistics - Simulation and Computation* 41, 10 (2012), 1928–1943. https://doi.org/10.1080/03610918.2011.626545 arXiv:https://doi.org/10.1080/03610918.2011.626545

[10] T.E. Oliphant. 2006. *A Guide to NumPy*. Trelgol Publishing.

[11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[12] Philipp Probst, Bernd Bischl, and Anne-Laure Boulesteix. 2018. Tunability: Importance of Hyperparameters of Machine Learning Algorithms. (2018). arXiv:arXiv:1802.09596 Preprint, https://arxiv.org/abs/1802.09596.

[13] Foster Provost and Pedro Domingos. 2000. Well-Trained PETs: Improving Probability Estimation Trees. (2000). *CDER working paper #IS-00-04, Stern Shool of Business, NYU, NY, NY, 10012.*

[14] Herbert A. Sturges. 1926. The Choice of a Class Interval. *J. Amer. Statist. Assoc.* 21, 153 (1926), 65–66. https://doi.org/10.1080/01621459.1926.10502161 arXiv:https://doi.org/10.1080/01621459.1926.10502161

[15] J. N. Van Rijn and F. Hutter. 2018. Hyperparameter Importance Across Datasets. *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2018). Preprint, https://arxiv.org/abs/1802.09596.

[16] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2014. OpenML. *ACM SIGKDD Explorations Newsletter* 15, 2 (jun 2014), 49–60. https://doi.org/10.1145/2641190.2641198

[17] Esteban Walker and Amy S. Nowacki. 2010. Understanding Equivalence and Noninferiority Testing. *Journal of General Internal Medicine* 26, 2 (sep 2010), 192–196. https://doi.org/10.1007/s11606-010-1513-8

A DATASETS

A.1 Properties OpenML-CC18

The OpenML-CC18 is a curated benchmark suite consisting of 73 classification tasks. For each dataset, the following properties hold:

- The number of observations is larger than 500;
- The number of observations is smaller than 100,000;
- The ratio of the minority class and the majority class is larger than 0.05;
- The number of values for categorical features most not exceed 100;
- The classification problem is not trivial (a model based on 1 feature does not result in perfect performance);
- Each target class contains at least 20 instances;
- The dataset does not belong to one of the following categories:
  - Artificial dataset
  - Simulated dataset
  - Time series dataset
  - Text data
  - Multilabel data
  - Derived versions of another dataset
  - Dataset where the intended classification target is unclear
  - Binarized regression problem
  - Dataset of unknown origin
  - Grouped data

A.2 Excluded tasks

The tasks of OpenML-CC18 that were not included in this study, due to either time constraints or technical issues, are listed in Table 7. Additionally, the number of features and number of instances of the datasets are shown in Figure 6. It is clear that the datasets that were excluded are all datasets with relatively many features or instances.

Table 7: OpenML-CC18 tasks excluded from this study. Tasks for which only the random forest or SVM timed out too often are annotated with respectively * and **.

OpenML task id	Reason
167125	technical issue
219**	time constraint
3481*	time constraint
3573	time constraint
7592	time constraint
9977	time constraint
14965**	time constraint
14970*	time constraint
146195	time constraint
146825	time constraint
167119**	time constraint
167120	time constraint
167121	time constraint
167124	time constraint

A.3 Datasets previous work

In Figure 7a we observe that both the study of Probst et al. [12] and Van Rijn and Hutter [15] contain more datasets with a large number of features and instances. In Figure 7c we observe that the data of Probst et al. [12] and Van Rijn and Hutter [15] contain relatively more

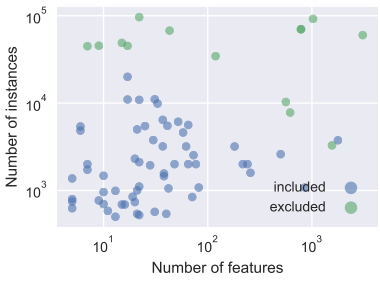
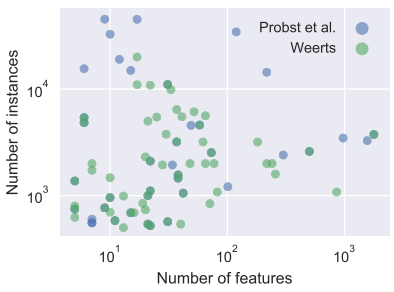
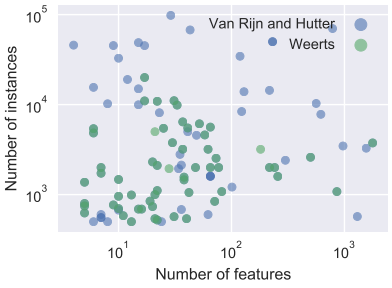


Figure 6: Number of features against number of instances of datasets of the OpenML-CC18 that were included and excluded from this study.

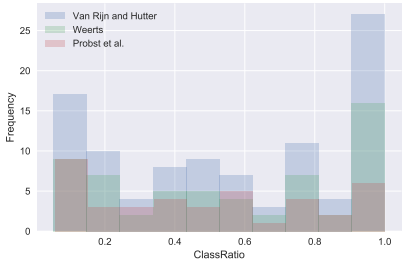
imbalanced dataset than ours. This makes sense, since these datasets are taken from the OpenML-100, for which the class imbalance was not yet one of the criteria (as opposed to the OpenML-CC18).



(a) The number of features against number of instances for this work and Probst et al. [12].



(b) The number of features against number of instances for this work and Van Rijn and Hutter [15].

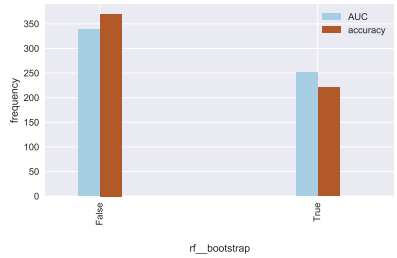


(c) Distribution of the ratio of the number of instances in the minority class compared to the majority class.

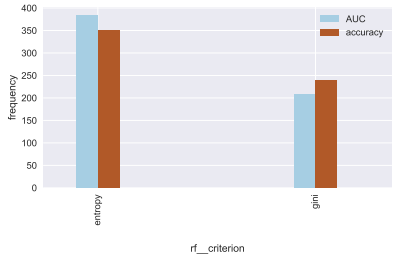
Figure 7: Meta-features for the datasets included in the work of Probst et al. [12], Van Rijn and Hutter [15] and this study.

B DISTRIBUTION OF TOP 10 PERFORMANCE DATA

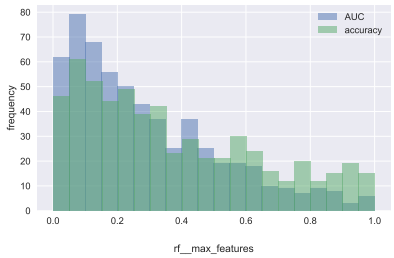
In Figure 8 histograms of the distribution of hyperparameters in the top 10 performance data are displayed for both accuracy and AUC.



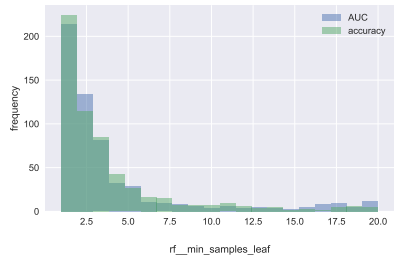
(a) Random Forest - `bootstrap`



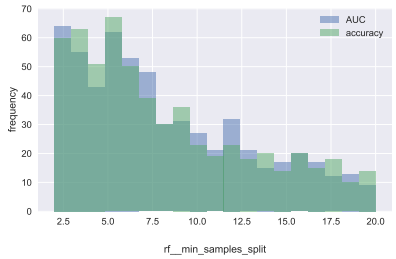
(b) Random Forest - `criterion`



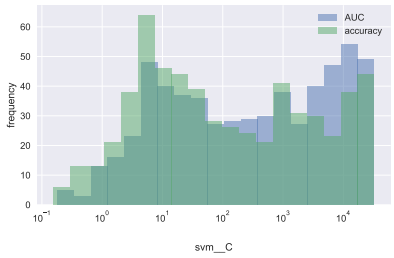
(c) Random Forest - `max_features`



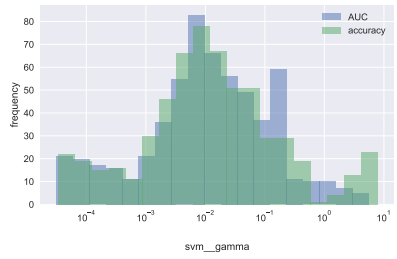
(d) Random Forest - `min_samples_leaf`



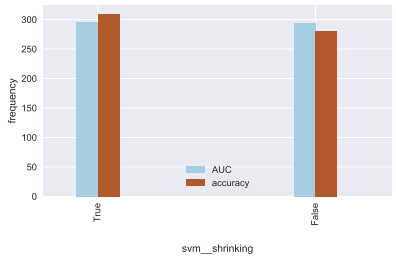
(e) Random Forest - `min_samples_split`



(f) SVM - `C`



(g) SVM - `gamma`



(h) SVM - `shrinking`

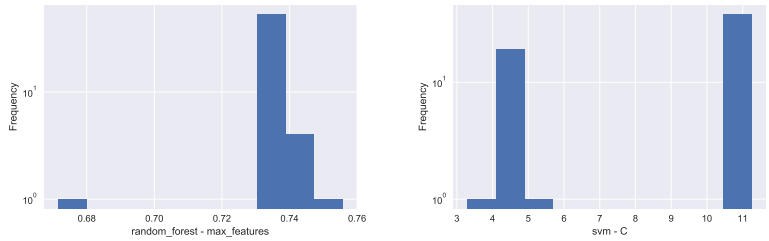


(i) SVM - `tol`

Figure 8: The distribution of a hyperparameters within the top 10 highest performing hyperparameter settings per dataset, measured by either accuracy or AUC.

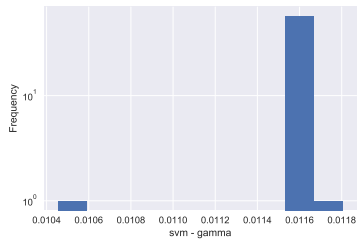
C DISTRIBUTION OF DEFAULT VALUES ACROSS TASKS

The distributions of the computed default values for all hyperparameters for which the standard deviation of the default values is non-zero is depicted in Figure 9 and Figure 10 derived from respectively accuracy and AUC-based performance data.

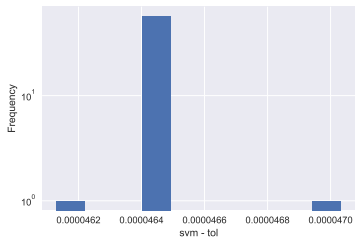


(a) Random Forest - `max_features`

(b) SVM - `criterion`

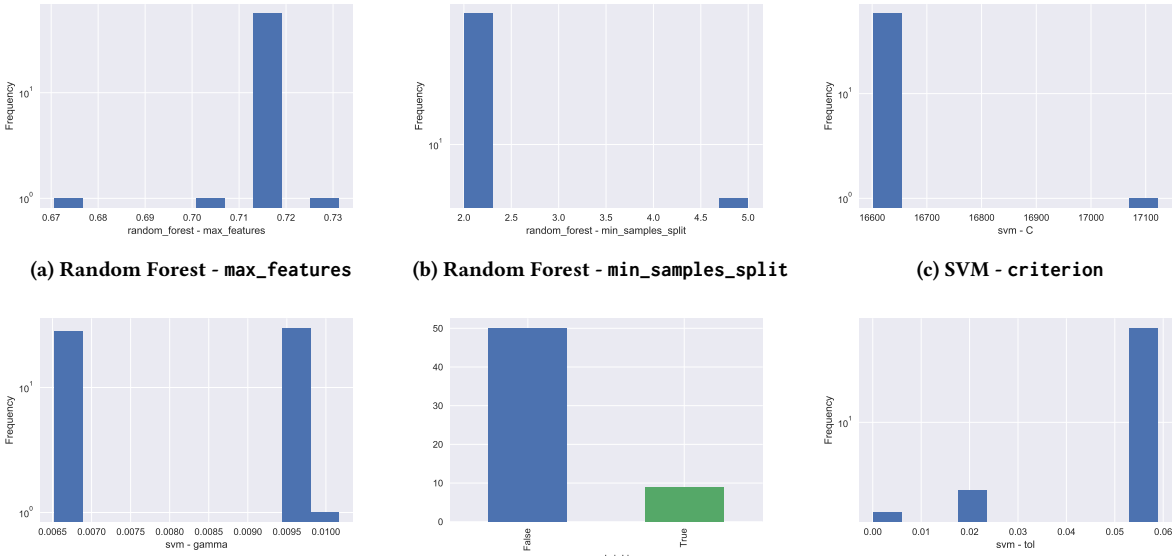


(c) SVM - `gamma`



(d) SVM - `tol`

Figure 9: The distribution of the computed default values, using accuracy based performance data, across different tasks for hyperparameters where the default values have a non-zero standard deviation.



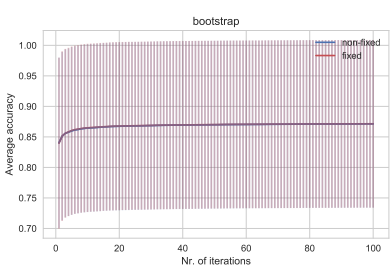
(a) Random Forest - `max_features` (b) Random Forest - `min_samples_split` (c) SVM - `criterion`

(d) SVM - `gamma` (e) SVM - `shrinking` (f) SVM - `tol`

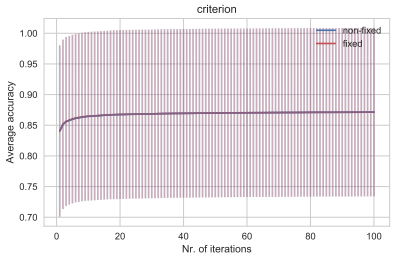
Figure 10: The distribution of the computed default values, using AUC based performance data, across different tasks for hyperparameters where the default values have a non-zero standard deviation.

D AVERAGE ACCURACY

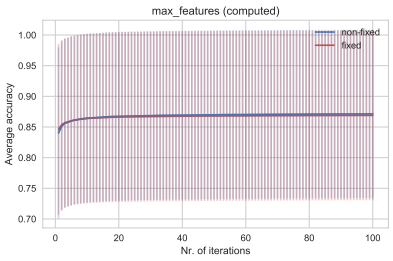
For each measurement, we computed the maximum average validation set accuracy up until a certain iteration. The average accuracy (+/- standard deviation) for both the fixed and non-fixed conditions is shown in Figure 11.



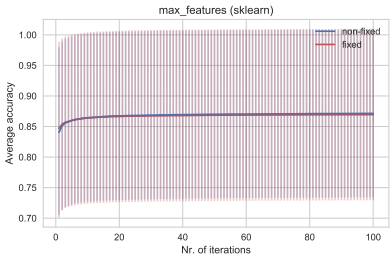
(a) Random Forest - `bootstrap`



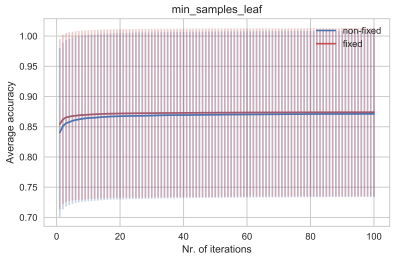
(b) Random Forest - `criterion`



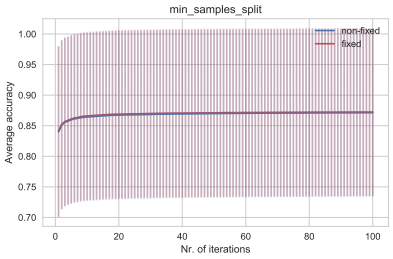
(c) Random Forest - `max_features` (computed default)



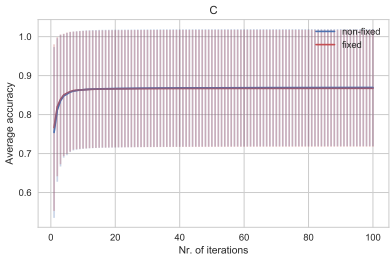
(d) Random Forest - `max_features` (scikit-learn default)



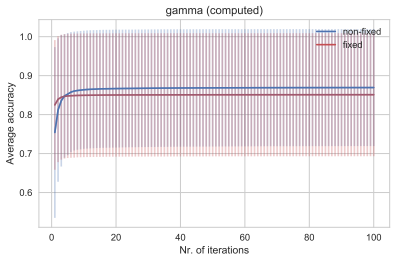
(e) Random Forest - `min_samples_leaf`



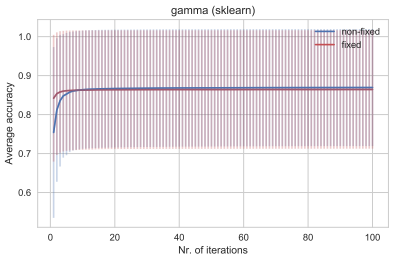
(f) Random Forest - `min_samples_split`



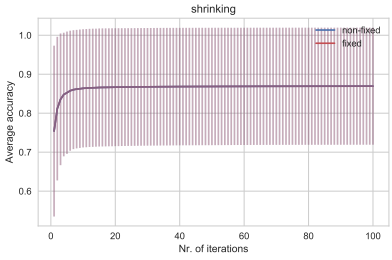
(g) SVM - `C`



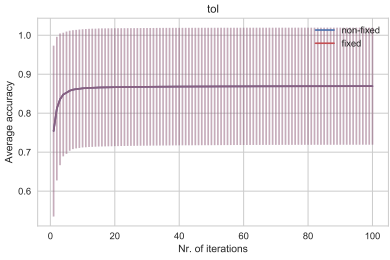
(h) SVM - `gamma` (computed default)



(i) SVM - `gamma` (scikit-learn default)



(j) SVM - `shrinking`

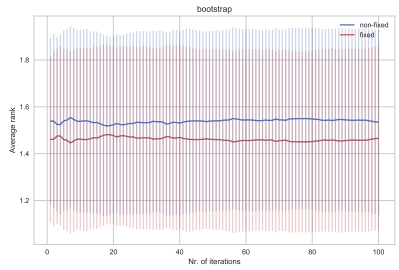


(k) SVM - `tol`

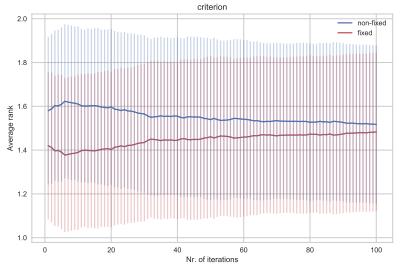
Figure 11: Average accuracy (+/- standard deviation) of the the fixed and non-fixed conditions across 59 datasets over number of iterations.

E AVERAGE RANK

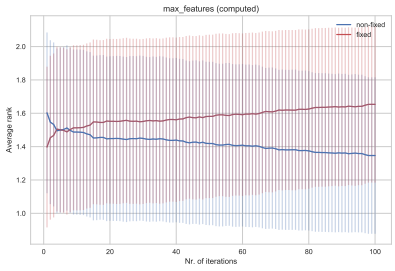
For each measurement, we computed the maximum average validation set accuracy up until a certain iteration. This data was used to rank the fixed and non-fixed conditions. The average rank (+/- standard deviation) is shown in Figure 12.



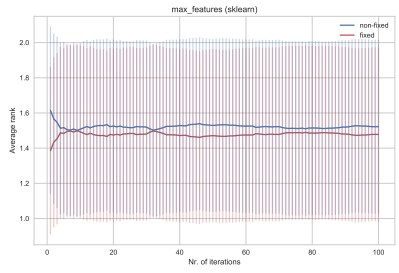
(a) Random Forest - `bootstrap`



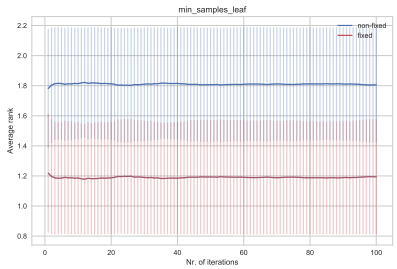
(b) Random Forest - `criterion`



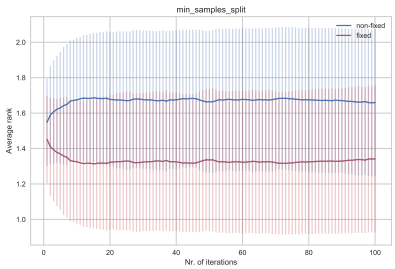
(c) Random Forest - `max_features` (computed default)



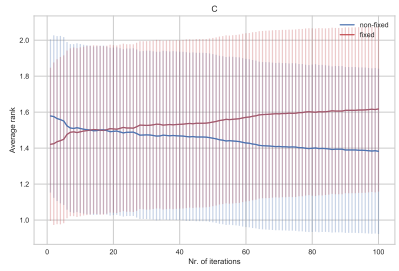
(d) Random Forest - `max_features` (scikit-learn default)



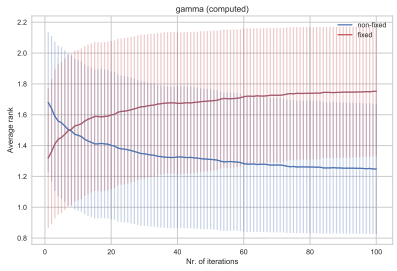
(e) Random Forest - `min_samples_leaf`



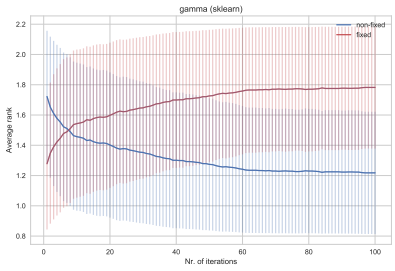
(f) Random Forest - `min_samples_split`



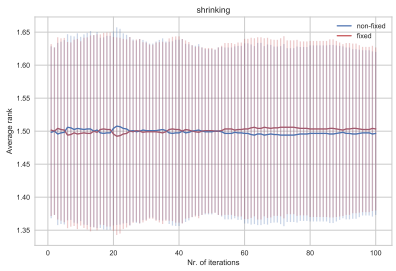
(g) SVM - `C`



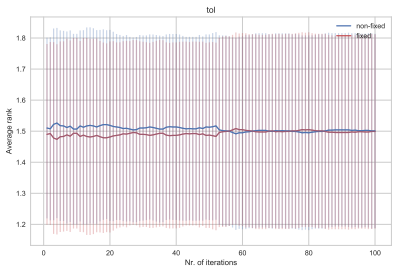
(h) SVM - `gamma` (computed default)



(i) SVM - `gamma` (scikit-learn default)



(j) SVM - `shrinking`



(k) SVM - `tol`

Figure 12: Average rank (+/- standard deviation) of the the fixed and non-fixed conditions across 59 datasets over number of iterations. Ranks are based on the maximum average validation set accuracy. Note that ranks are a relative measure of performance and that smaller is better.